

Network Metrics

Prof. Christiancarmine Esposito
esposito@unisa.it

Maria Caterina D'Aloia
mdaloia@unisa.it

Agenda



Evaluation Metrics

Fidelity, entanglement rate, QBER, throughput, latency, secret key rate.
How to measure them and what they tell us about network health.



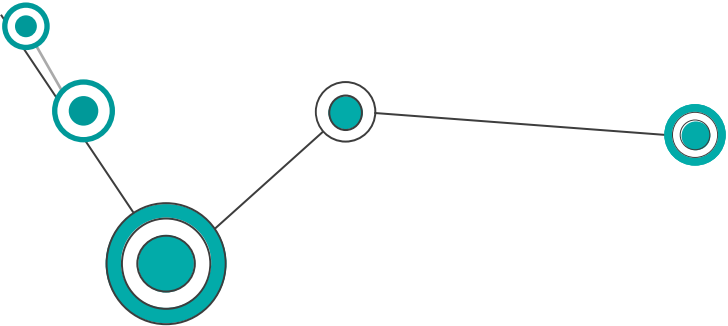
Metrics as Attack Signals

How anomalous variations in quantum network metrics can reveal active and passive attacks. Examples from the research literature.



SquidASM

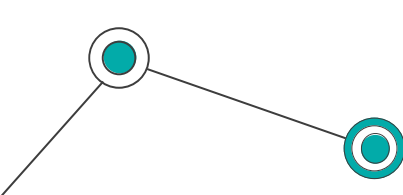
Framework for distributed quantum applications on simulated or real networks. Architecture, installation, code example.



PART A

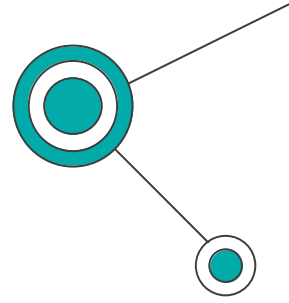
Network Evaluation Metrics

How to measure the performance of a quantum network



Why Metrics?

The Challenge of Quantum Quality



In a classical network you just measure bit rate and latency. In a quantum network, signal quality is entanglement quality.

Problem 1

You cannot copy or measure a qubit without destroying it (no-cloning, no-monitoring)

Problem 2

Entanglement degrades over time — the channel may be good now but decayed by the time it is used

Problem 3

Gate errors, photon loss, decoherence: every noise source accumulates across the network

Solution

Measure statistically over many copies — extract aggregate metrics without disturbing the signal

Fidelity

$$F = \langle \psi_{\text{ideal}} | \rho_{\text{real}} | \psi_{\text{ideal}} \rangle$$

range: $[0, 1]$
threshold

$F > 0.5$ = above classical

What it measures

The overlap between the real quantum state ρ and the ideal target $|\psi\rangle$.

F = 1 → perfect state, no noise.

F = 0.5 → classical threshold (separability limit).

F < 0.5 → unusable state, entanglement lost.

Computing fidelity in NetSquid

```
import netsquid as ns
from netsquid.qubits import qubitapi as qapi

# Create the ideal reference Bell state
q_ref, = qapi.create_qubits(1)

# Compute fidelity against the current real state
fidelity = qapi.fidelity([q_real], reference_state,
squared=True)
print(f"Fidelity: {fidelity:.4f}")

# In simulation: computed directly from the density matrix.
# On hardware: requires quantum state tomography over many
shots.
```

QBER (Quantum Bit Error Rate)

$$\text{QBER} = \frac{N_{\text{errors}}}{N_{\text{total_measurement}}}$$

Safe threshold for BB84: $\text{QBER} < 11\%$

WHAT IS QBER

The fraction of bits measured in the quantum channel that differ from the expected value.

Computed by comparing Alice's and Bob's results in matching measurement bases (sifting).

Tracked separately per basis: QBER_X and QBER_Z.

THRESHOLDS & INTERPRETATION

$\text{QBER} < 11\% \rightarrow$ channel is secure for BB84

$\text{QBER} \approx 11\% \rightarrow$ theoretical security limit

$\text{QBER} > 25\% \rightarrow$ channel is completely noisy

In NetSquid, computed by the DataCollector over many runs:

$Q_x = \text{X-basis errors} / \text{X-basis measurements}$

$R_{SK} = \max(0, 1 - H(Q_x) - H(Q_z)) / 2 \cdot AS$

QBER is simultaneously a quality metric and a security indicator: an abnormally high QBER signals noise — or an attack.

Entanglement Rate and Throughput

ENTANGLEMENT RATE

Definition:

Number of successfully distributed entangled pairs per unit time [pairs/s].

Depends on:

- Generation success probability
- Propagation delay in fiber
- BSM and gate duration
- Number of required retries

Simplified formula:

$$R = p_{\text{success}} / (2 \cdot L/c + t_{\text{BSM}})$$

THROUGHPUT / SECRET KEY RATE

Secret Key Rate (SKR):

Secret key bits generatable per distributed entangled pair.

$$R_{\text{SK}} = \max(0, 1 - H(Q_x) - H(Q_z)) / 2 \cdot AS$$

where AS = average attempts per success.

Effective throughput:

Combines entanglement rate and SKR:

$$T = \text{Entanglement_Rate} \times \text{SKR} \quad [\text{bit/s}]$$

Latency and Coherence Time

DISTRIBUTION LATENCY

Definition

Time from source trigger to confirmed heralding received by both nodes.

Components

$t_{\text{propagation (fiber)}} + t_{\text{BSM}} + t_{\text{classical_heralding}} + t_{\text{wait (if slot busy)}}$

In NetSquid

Measured with `ns.sim_time()` before trigger and after `rx_input()` of heralding.
Typical: $2 \times L/c + t_{\text{gate}}$

Relevance

Directly sets the time window in which qubits must survive in memory (must be $< T_2$).

COHERENCE TIME (T_1 , T_2)

Definition

T_1 = energy relaxation time. T_2 = dephasing time (phase coherence loss).

Network impact

If latency exceeds T_2 , the qubit is decoherent before use $\rightarrow$ fidelity collapses.

Technologies

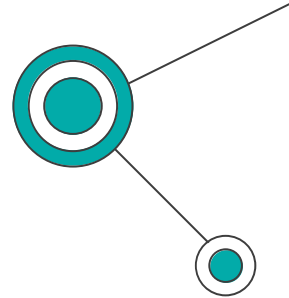
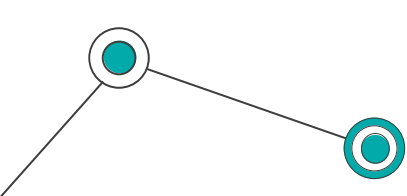
NV center: $T_2 \sim 1 \text{ s}$ (^{13}C nuclear). Ion trap: $T_2 \sim$ minutes. Superconducting: $T_2 \sim \mu\text{s}$.

In simulation

Modelled by `T1T2NoiseModel` or dephasing channel applied retroactively on qubit access.

Summary

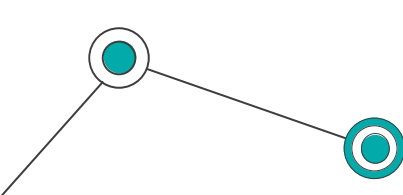
Metric	Symbol	Typical range	Critical threshold	What it measures
Fidelity	F	0 – 1	$F > 0.5$	Quality of the entangled state
QBER	Q	0% – 50%	$Q < 11\%$	Errors in the quantum channel
Entanglement Rate	R	Hz – MHz	$f(\text{distance})$	Speed of entanglement distribution
Secret Key Rate	R_{SK}	bit/s	> 0	Generatable secure key bits
Latency	Δt	$\mu\text{s} - \text{ms}$	$< T_2$	Time from generation to delivery
Coherence time	T_2	$\mu\text{s} - \text{s}$	hardware	Useful lifetime of qubit in memory



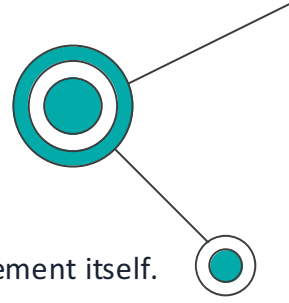
PART B

Metrics as Attack Signals

How the literature uses metrics to detect intrusions



Attack Types on Quantum Networks



Unlike classical networks, a quantum attacker cannot copy data in transit — but can interfere with the entanglement itself.

Intercept-Resend

Eve intercepts Alice's qubit, measures it, then forwards a copy to the Repeater.

Measurable effect: QBER rises sharply (up to 25% in BB84).

Man-in-the-Middle (Entanglement)

Eve inserts herself between Alice and the Repeater, generating separate entanglement with each.

Measurable effect: fidelity drops, rate behaves anomalously.

Photon Number Splitting (PNS)

On multi-photon sources: Eve retains one photon without detection.

Measurable effect: QBER stays low but the key is compromised — rate drops.

Denial of Service (Decoherence)

Eve injects noise into the channel (electromagnetic interference) to degrade T_2 .

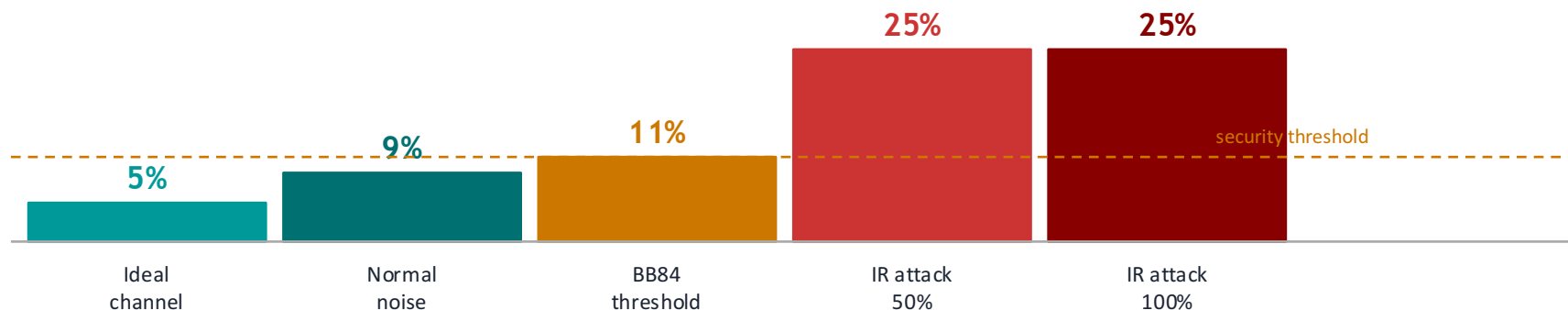
Measurable effect: fidelity drops, rate collapses, latency increases.

QBER as Detector – The Intercept-Resend Case

Theory (BB84 – Bennett & Brassard, 1984):

Without attack: QBER from channel noise only → typically 1–5%.

With full Intercept-Resend: Eve measures in the wrong basis 50% of the time → introduces 25% errors into QBER.



Literature

Scarani et al. (2008)

QBER + SKR

Security of Practical Quantum Key Distribution

Any individual attack on BB84 is detectable through a QBER rise above 11%. The SKR drops to 0 above threshold → system halts automatically.

Makarov et al. (2005)

Fidelity + rate

Faked-State Attack on Quantum Cryptosystem

An attacker exploiting detector flaws can keep QBER normal while fidelity behaves anomalously. They propose detection-state fidelity as a countermeasure.

Coopmans et al. (NetSquid, 2021)

Fidelity vs. hw
params

NV Repeater Chain Sensitivity

In simulation: detection probability is the most critical parameter. Degrading it simulates passive attacks (photon blocking). Fidelity < 0.5 = unusable channel.

Pirandola et al. (2019)

PLOB bound + rate

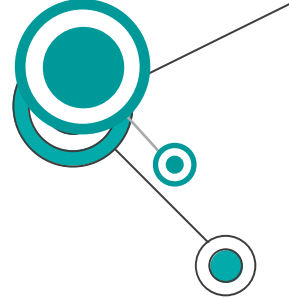
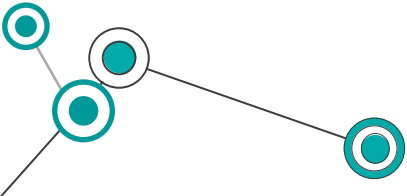
Advances in Quantum Computing

Entanglement rate decays exponentially with distance. A faster-than-theoretical decay signals anomalous loss (PNS attack or active interference).

Anomaly Fingerprint

Every type of disruption leaves a recognisable signature across the metric profile:

Event type	QBER	Fidelity	Rate	Latency	T ₂ eff.
Intercept-Resend	↑ high (>11%)	↓	↓	→	→
PNS Attack	→ (masked)	→	↓ slight	→	→
Man-in-Middle	↑ slight	↓↓	anomalous	↑	→
Decoherence/DoS	↑	↓↓	↓↓	→	↓ simul.
No attack	< 5%	> 0.85	nominal	nominal	nominal



PART C

SquidASM

Distributed quantum applications on real quantum networks

What is SquidASM?

SquidASM = NetSquid + NetQASM → execution of quantum programs on simulated or real network nodes

Developed at: QuTech, Delft. Built on NetSquid + the NetQASM language for programming individual nodes.

Goal: Allow developers to write distributed quantum applications (QKD, quantum teleportation, distributed algorithms) using a high-level interface, testable in simulation (NetSquid) or on real hardware (Quantum Network Explorer — QNE).

NetSquid

Simulates network physics. Components, channels, noise models, discrete events. Hardware level.

NetQASM

Assembly language for programming a single quantum node. Instructions: `create_epr`, `recv_epr`, `measure`.

SquidASM

Combines both: executes NetQASM programs on NetSquid nodes. Tests distributed protocols end-to-end.

Installing SquidASM

Step 1 – Prerequisites: Python 3.8+, NetSquid installed (see Lesson 1), netsquid.org account

```
# Step 2 – Install SquidASM from public PyPI
pip install squidasm
```

```
# Dependencies installed automatically:
#   netsquid, netqasm, numpy, scipy
```

```
# Step 3 – Verify the installation
import squidasm
print(squidasm.__version__)      # e.g. 0.12.1
```

```
# Step 4 – Import the main modules
from squidasm.run.stack.run import run
from squidasm.sim.stack.program import Program, ProgramContext, ProgramMeta
from netqasm.sdk.classical_communication.message import StructuredMessage
```

SquidASM Architecture — How It Works

Application (Python)	AliceProgram / BobProgram — QKD, teleportation, distributed algorithm logic
SquidASM SDK	Program, ProgramContext, epr_sockets, csockets — high-level interface
NetQASM	Instructions: create_epr, recv_epr, measure, gate_x, gate_h — quantum assembly
NetSquid Simulation	Node, QSource, QuantumProcessor, QuantumChannel — physical simulation layer
PyDynAA	Discrete-event engine — timeline, events, scheduling (shared with NetSquid)

↑ high level

↓ low level

run() executes all Programs in parallel on separate NetSquid nodes, synchronised by the discrete-event engine.

BB84 QKD with SquidASM

```
class AliceProgram(Program):
    def run(self, context: ProgramContext):
        epr_socket = context.epr_sockets["bob"]
        csocket      = context.csockets["bob"]
        conn         = context.connection

        qubit, = epr_socket.create_keep(1)           # generate EPR pair, keep 1 qubit
        basis = random.randint(0, 1)                 # choose random basis (0=Z, 1=X)
        if basis == 1:
            qubit.H()                                # switch to X basis
        result = qubit.measure()                     # measure the qubit
        yield from conn.flush()                       # execute circuit on NetSquid node
        csocket.send(StructuredMessage("basis", basis)) # announce basis
        bob_basis = yield from csocket.recv()         # receive Bob's basis
        if bob_basis.payload == basis:                 # sifting: keep only matching bases
            sifted_key.append(int(result))
        return {"sifted_key": sifted_key, "raw_result": int(result)}
```

create_keep(1): generates an EPR pair and keeps the local qubit. **H():** Hadamard gate for basis change. **flush():** sends the queued circuit to the NetSquid node for execution.

To run: `run(config, programs={"alice": AliceProgram(), "bob": BobProgram()})` – automatically parallelised.

SquidASM — Collecting Network Metrics

SquidASM returns structured results for each node. From these we compute QBER, rate, and SKR:

```
# Run the simulation and collect results
results = run(config=cfg, programs={"alice": AliceProgram(), "bob": BobProgram()}, num_times=1000)

alice_results = results[0]["alice"]  # list of dicts, one per run
bob_results   = results[0]["bob"]

# Compute QBER
errors = sum(1 for a, b in zip(alice_results, bob_results)
             if a["sifted_key"] != b["sifted_key"] and a["sifted_key"] is not None)
total  = sum(1 for a in alice_results if a["sifted_key"] is not None)
QBER   = errors / total if total > 0 else 0
print(f"QBER: {QBER:.4f} ({QBER*100:.2f}%)")

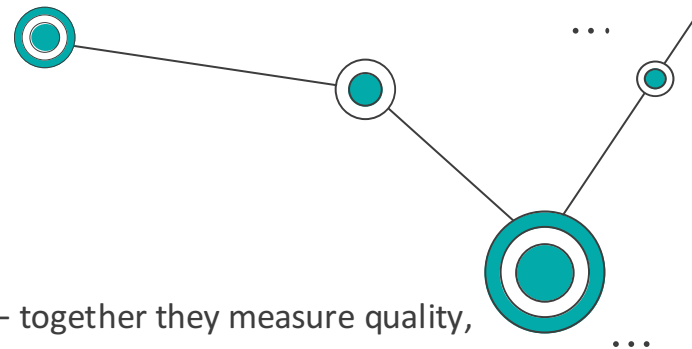
# Compute Secret Key Rate
def H(p): return -p*log2(p) - (1-p)*log2(1-p) if 0<p<1 else 0
SKR = max(0, 1 - H(QBER) - H(QBER)) / 2  # simplified BB84 formula
print(f"Secret Key Rate: {SKR:.4f} bit/pair")
```

num_times=1000: one thousand independent parallelisable runs. **results[0]:** result list indexed by program name. **H(p):** binary entropy for the SKR formula.

Setting depolar_rate > 0 on the channel simulates an attack — QBER rises above threshold, exactly as theory predicts.

NetSquid vs SquidASM – When to Use Which

Aspect	NetSquid	SquidASM
Level	Hardware (physical)	Application (logical)
Language	Python + NetSquid API	Python + NetQASM
Abstraction	Qubits, ports, physical gates	EPR sockets, csockets
Ideal for	Model hardware, noise, topologies	Develop & test QKD protocols
Complexity	High — requires manual wiring	Medium — ready-made templates
Compatibility	Simulation only	Simulation + QNE hardware
Metrics output	sim_stats, DataCollector	Structured results dict



KEY TAKEAWAYS — LESSON 2

- ① 6 key metrics: Fidelity, QBER, Entanglement Rate, SKR, Latency, T_2 — together they measure quality, speed, and security of a quantum network.
- ② Metrics as attack detectors: QBER > 11% signals interception. Anomalous fidelity indicates a man-in-the-middle. A collapsing rate signals PNS or DoS. Every attack leaves a distinct fingerprint.
- ③ SquidASM: Framework for writing distributed protocols at high level (NetQASM) and testing them on NetSquid or on real hardware (QNE). Perfect for QKD and teleportation.

